Fundamental Programming in JAVA

EXCEPTION HANDLING

Objectives

- To get an overview of exceptions and exception handling
- To explore the advantages of using exception handling.
- To distinguish exception types: Error (fatal) vs. Exception (nonfatal) and checked vs. unchecked.
- To declare exceptions in a method header.
- To throw exceptions in a method.
- To write a try-catch block to handle exceptions.
- To use the finally clause in a try-catch block.

Exception-Handling Overview

- When a program runs into a runtime error, the program **terminates abnormally**. How can you handle the runtime error so that the program can continue to run or terminate gracefully? This is the subject we will introduce in this chapter.
- What is an exception?
  - In Java, an exception is an event that occurs during the execution of a program and disrupts its normal flow.
  - It represents an abnormal condition or error situation that needs to be handled.
  - When an exceptional condition occurs, an exception object is created and thrown, which can be caught and handled by appropriate code.

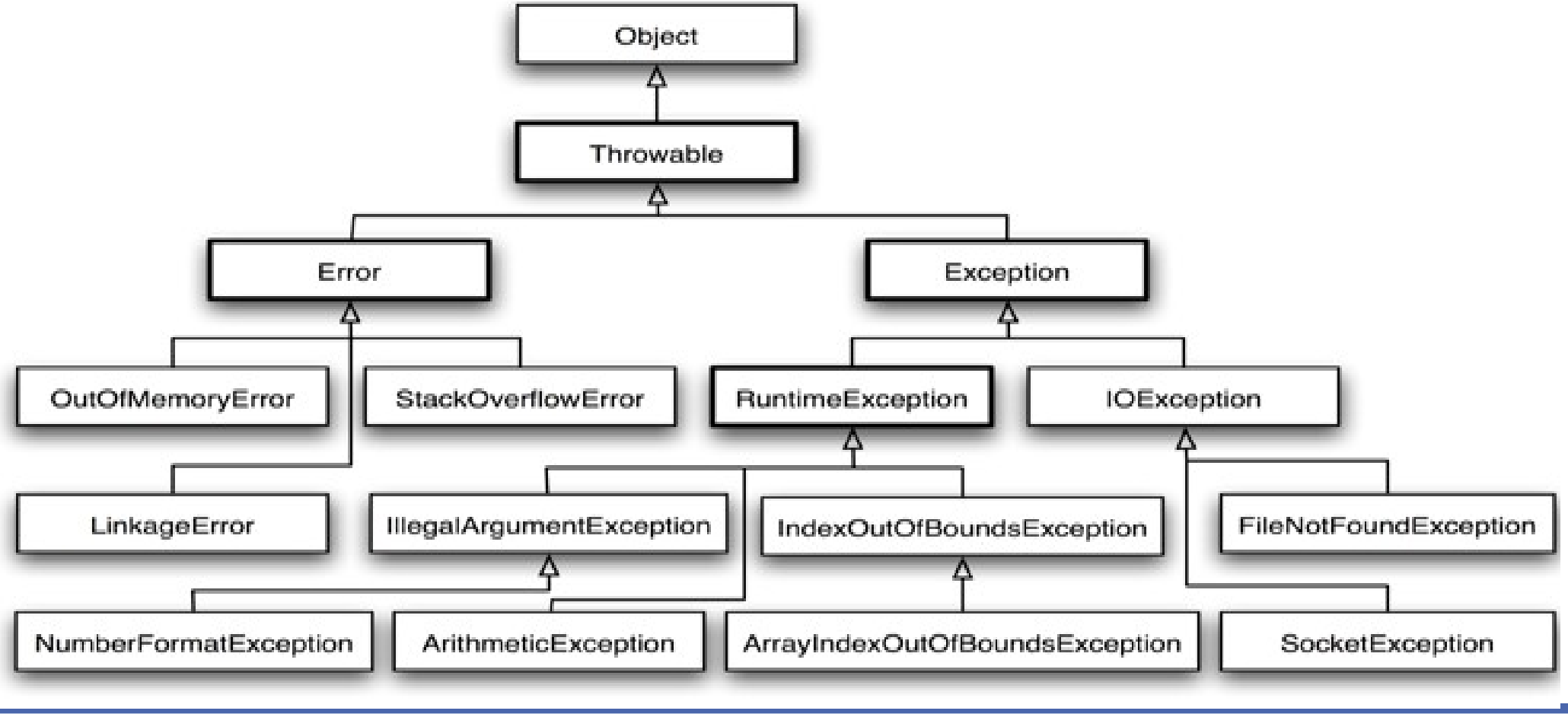
Exception-Handling Overview

- What is an exception?
  - Exceptions are the objects representing the logical errors that occur at run time and makes JVM enters into the state of "ambiguity".
  - The objects which are automatically created by the JVM for representing these run time errors are known as Exceptions.
  - An exception can occur for many different reasons, including the following:
    - A user has entered invalid data.
    - A file that needs to be opened cannot be found.
    - A network connection has been lost in the middle of communications or the JVM has run out of memory
  - Some of these exceptions are caused by user error, others by programmer error, and others by physical resources that have failed in some manner.

Exception-Handling Overview

- When an error occurs within a method, the method creates an object and hands it off to the runtime system. The object, called an exception object, contains information about the error, including its type and the state of the program when the error occurred. Creating an exception object and handing it to the runtime system is called throwing an exception

Exception Types

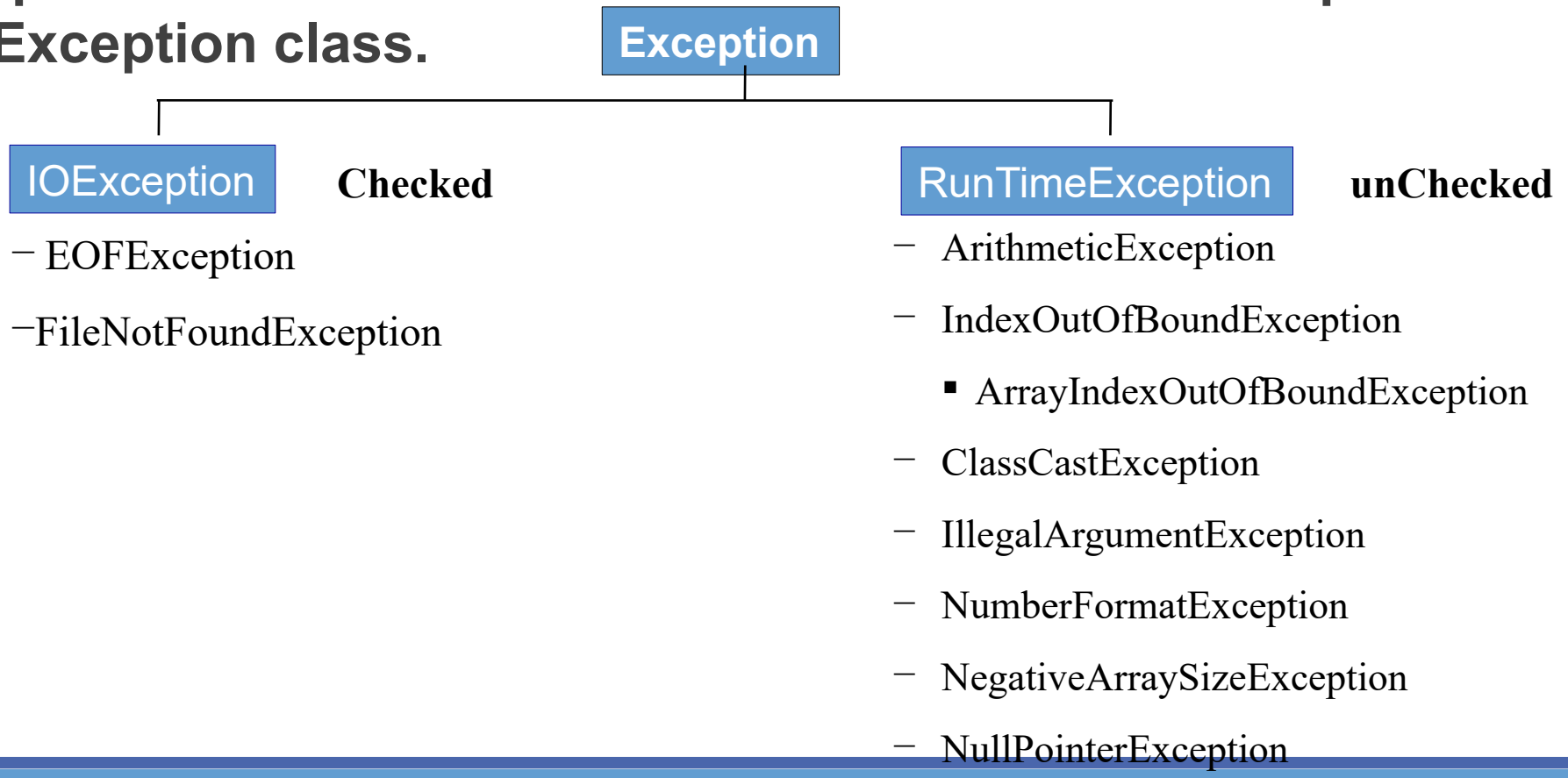


Exception Types

- System errors are thrown by JVM and represented in the Error class. The Error class describes internal system errors. Such errors rarely occur. If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.
- Exception describes errors caused by your program and external circumstances. These errors can be caught and handled by your program.

Checked Exceptions vs. Unchecked Exceptions

- **The Exception class has two main subclasses: the IOException class and the RuntimeException class.**



Checked Exceptions vs. Unchecked Exceptions

- RuntimeException, **Error** and their **subclasses** are known as **unchecked exceptions**. **All other exceptions are known as checked exceptions**, *meaning that the compiler forces the programmer to check and deal with the exceptions*.

Unchecked Exceptions

- In most cases, unchecked exceptions reflect programming logic errors that are not recoverable. For example, a NullPointerException is thrown if you access an object through a reference variable before an object is assigned to it; an IndexOutOfBoundsException is thrown if you access an element in an array outside the bounds of the array. These are the logic errors that should be corrected in the program.
- For Unchecked Exceptions, the *compiler doesn't check during the compilation process*. Hence, it isn't mandatory for the method to handle these exceptions. All unchecked exceptions extend the class RuntimeException.

Exceptions Classes Example

- In this short introductory note on exceptions, we will concentrate on runtime exceptions ,example1 :

```java
1  public class Try1 {
2
3      public static void main(String[] args) {
4          int a = 5;
5          int b = 0;
6          System.out.println(a/b);
7      }
8  }
```

```
Output - Exception1 (run)
run:
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at Try1.main(Try1.java:6)
Java Result: 1
BUILD SUCCESSFUL (total time: 2 seconds)
```

Exceptions Classes Example

- Example 2

```java
public class Try1 {

    public static void main(String[] args) {
        String[] countries = {"USA", "Canada", "Mexico", "Japan"};
        System.out.println(countries[10]);
    }
}
```

Output - Exception1 (run)

```
run:
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 10
        at Try1.main(Try1.java:5)
Java Result: 1
BUILD SUCCESSFUL (total time: 0 seconds)
```

Exceptions Classes Example

- example3 :

```java
1  public class Try1 {
2
3      public static void main(String[] args) {
4          Object[] obj = new Object[10];
5          obj[0].toString();
6
7      }
8  }
```

Output - Exception1 (run)

```
run:
Exception in thread "main" java.lang.NullPointerException
        at Try1.main(Try1.java:5)
Java Result: 1
BUILD SUCCESSFUL (total time: 0 seconds)
```

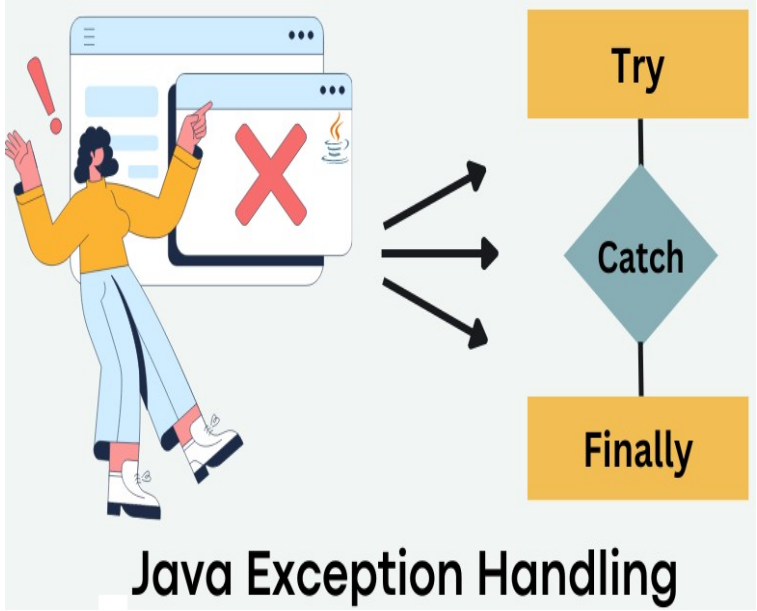
Most Common Java Exceptions

- **NullPointerException** : occurs when the program uses the reference of the object set to null.
- **ArrayIndexOutOfBoundsException** : occurs when an array is accessed with an illegal index.
- **StringIndexOutOfBoundsException**: occurs when an attempt is made to access the character of a string at an index which is either negative or greater than the length of the string.
- **NumberFormatException** : Java allows the conversion of String to numeric types. if the String doesn't have an appropriate format during the conversion, the method throws a NumberFormatException
- **ClassCastException** : at runtime, if the code attempts to downcast an object to a subtype of which it isn't an instance, the method throws a ClassCastException.
- **ArithmeticException** : For instance, if we try to divide an integer by zero, we get an ArithmeticException:

Catching Exceptions

- A method catches an exception using a combination of the **try** and **catch** keywords. A try/catch block is placed around the code that might generate an exception. Code within a try/catch block is referred to as **protected code**, and the syntax for using try/catch looks like the following:

```
try {
    // your code
} catch (FirstException ex) {
    // your code for handling the occurrence of FirstException
} catch (SecondException ex) {
    // your code for handling the occurrence of SecondException
} finally { .... }
```



Try and Catch: Syntax

- The try block :

```
Try{
 // your code
}
```

- If an exception occurs within the try statement, that exception is handled by the appropriate exception handler associated with this try statement. A try statement must be accompanied by at least one catch block or one final block.

Try and Catch: Syntax

- **The catch block** : You associate exception handlers with a try statement by providing one or more catch blocks directly after the try block:

```
try { // your code}
catch (….) { …….}
catch (….) { …… }
```

- The general form is:

```
catch (SomeException variableName) {
    // your exceptional handler
    // codes for handling the occurrence of Some Exception
}
```

Try and Catch: Syntax

- **The catch block** :The catch block contains a series of legal Java statements, and these statements are executed if the exception handler is invoked.
- The Steps are :
  - when an exception occurs in the try block.
  - The exception matches an exception declared in the catch.
  - The code inside the catch block will be executed.
  - After the statements inside the catch block are executed, the flow of the program leaves the whole try-catch block.

Try and Catch: Syntax

- The finally block :
  - The finally keyword is used to create a block of code that follows a try block.
  - A finally block of code always executes, whether or not an exception has occurred.
  - Using a finally block allows you to run any cleanup-type statements that you want to execute, no matter what happens in the protected code.
  - A finally block appears at the end of the catch blocks and has the following syntax (see next slide).

Try and Catch: Syntax

```
try {
    // your code
} catch (FirstException ex) {
    // your code for handling the occurrence of FirstException
} catch (SecondException ex) {
    // your code for handling the occurrence of SecondException
catch (SecondException ex) {
    // your code for handling the occurrence of SecondException
} finally {
  //The finally block always executes }
```

Try and Catch: Steps

- Finally will be executed in all cases.
- If no errors happen……. The catch will not be executed.
- But, if an error happens and there is a catch statement that can catch the thrown exception:
  - Stop executing the try block
  - Go directly to catch block
  - Search them one by one until finding the proper exception (The same exception or a superclass from it)
  - Execute the finally block (if it exists)
  - Complete execution after the finally block
- If an error happens and there is no catch statement that can catch the thrown exception: Propagate back to the calling methods with uncaught exception until reach a proper catch statement or JVM.

Try and Catch: Example

```java
public class Try1 {

    public static void main(String[] args) {
        int a = 5;
        int b = 0;
        try {
            System.out.println(a / b);
        } catch (ArithmeticException ex) {
            System.out.println("You can't divide a number by zero!");
        }
        System.out.println("Program ends");
    }
}
```

Output - Exception1 (run)

```
run:
You can't divide a number by zero!
Program ends
BUILD SUCCESSFUL (total time: 1 second)
```

1. a/b is 5/0. An ArithmeticException occurs.
2. Search through the catch blocks. The first one (and the only one) matches the exception. So the codes in the block are executed.
3. After the codes are executed. The next code is the code outside the whole try-catch block.

Try and Catch: Example

```java
public class Try1 {
    public static void main(String[] args) {
        String[] countries = {"USA", "Canada", "Mexico", "Japan"};
        try {
            System.out.println(countries[10]);
        } catch (ArrayIndexOutOfBoundsException ex) {
            System.out.println("Array index out of bounds");
        } catch (NullPointerException ex) {
            System.out.println("Null pointer");
        }
    }}
```

Output - Exception1 (run)

```
run:
Array index out of bounds
BUILD SUCCESSFUL (total time: 1 second)
```

1. ArrayIndexOutOfBoundsException occurs in the try block.
2. compare the exceptions in the catch blocks. The first one is matched.
3. The codes inside the catch block are executed.

Try and Catch: Example

```
1   public class Try1 {
2       public static void main(String[] args) {
3           Object[] obj = new Object[10];
4           try {
5               System.out.println(obj[0].toString());
6           } catch (ArrayIndexOutOfBoundsException ex) {
7               System.out.println("Array index out of bounds");
8           } catch (NullPointerException ex) {
9               System.out.println("Null pointer");
10          }
11      }}
```

Output - Exception1 (run)

```
run:
Null pointer
BUILD SUCCESSFUL (total time: 0 seconds)
```

Exception Methods

The following are the most important methods available in the Throwable class:

getMessage()

public String getMessage(){ }

Description: Returns a detailed message about the exception that has occurred. This message is initialized in the Throwable constructor.

printStackTrace()

public void printStackTrace (){ }

Description: is a tool used to handle exceptions and errors. It is a method of Java's throwable class that prints the throwable along with other details like the line number and class name where the exception occurred.

Exception Methods

```
1   public class Try1 {
2       public static void main(String[] args) {
3           int a = 5;
4           int b = 0;
5           try {
6               System.out.println(a / b);
7           } catch (ArithmeticException ex) {
8               System.out.println(ex.getMessage());
9               ex.printStackTrace();
10          }
11          }}
```

Output - Exception1 (run)

```
run:
/ by zero
java.lang.ArithmeticException: / by zero
        at Try1.main(Try1.java:6)
BUILD SUCCESSFUL (total time: 1 second)
```

Try and Catch: Example

```java
public class Try1 {
    public static void main(String[] args) {
        int a[] = new int[2];
        try {
            System.out.println("Access element three :" + a[3]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Exception thrown  :" + e);
        } finally {
            a[0] = 6;
            System.out.println("First element value: " + a[0]);
            System.out.println("The finally statement is executed");
        }
```

Output - Exception1 (run)

```
run:
Exception thrown  :java.lang.ArrayIndexOutOfBoundsException: 3
First element value: 6
The finally statement is executed
BUILD SUCCESSFUL (total time: 1 second)
```

Try and Catch: Notes

- A catch clause cannot exist without a try statement.
- It is not compulsory to have finally clauses whenever a try/catch block is present. **(finally is an optional block)**
- The try block cannot be present without either a catch clause or a finally clause.
- Any code cannot be present in between the try, catch, and finally blocks.

Note: the order of the catch statement is crucial, why? The following catch can't be the first one, it could be the last one.

```
catch (Exception ex) {
    // Handle all other exception types.

    System.out.println(ex.getMessage());

    ex.printStackTrace();
}
```

← **Explain**

← **Explain**

References

- [1] Introduction to Java Programming, Comprehensive Version, Student Value Edition, Daniel Liang, 12th Edition, Pearson 2019
- lecture Notes :edugate.asu.edu.jo.
- lecture Videos : YouTube channel.
- Lab manual ???